



CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 3

Topic : Round Robin (Preemptive)

Presented by,

Mashrur Rahman Masfi
Reg : 23101182
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

Consider the following processes, each represented as (Process ID, Arrival Time, Burst Time). The **time quantum is 5 units**. Using the preemptive **Round Robin** scheduling algorithm, calculate: Completion Time (CT), Turnaround Time ($TAT = CT - AT$), Waiting Time ($WT = TAT - BT$), and the average TAT and WT.

Process ID	Arrival Time (A.T)	Burst Time (B.T)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

Solution (step-by-step simulation, TQ = 5):

Step	Time	Process	Executed	Remaining BT	Queue after execution
1	0-5	P1	5	2	P2,P3,P4,P5,P6,P ₁
2	5-9	P2	4	0	P3,P4,P5,P6,P1
3	9-14	P3	5	10	P4,P5,P6,P1,P3
4	14-19	P4	5	6	P5,P6,P1,P3,P4
5	19-24	P5	5	15	P6,P1,P3,P4,P5
6	24-29	P6	5	4	P1,P3,P4,P5,P6
7	29-31	P1	2	0	P3,P4,P5,P6
8	31-36	P3	5	5	P4,P5,P6,P3
9	36-41	P4	5	1	P5,P6,P3,P4
10	41-46	P5	5	10	P6,P3,P4,P5
11	46-50	P6	4	0	P3,P4,P5
12	50-55	P3	5	0	P4,P5
13	55-56	P4	1	0	P5
14	56-61	P5	5	5	P5
15	61-66	P5	5	0	-

Source Code :

Round Robin (preemptive)-

```

1  def round_robin(parameters, quantum):
2      n = len(parameters)
3      pid = [p[0] for p in parameters]
4      arrival = [p[1] for p in parameters]
5      burst = [p[2] for p in parameters]
6      remaining = burst.copy()
7
8      completion = [0] * n
9      turnaround = [0] * n
10     waiting = [0] * n
11     response = [0] * n
12     first_run = [-1] * n
13
14     order = sorted(range(n), key=lambda i: (arrival[i], pid[i]))
15     queue = []
16     gantt = []
17     current_time = 0
18     completed = 0
19     ptr = 0
20
21     while completed < n:
22         # CPU idle: jump to next arrival
23         if not queue:
24             current_time = max(current_time, arrival[order[ptr]])
25             queue.append(order[ptr])
26             ptr += 1
27
28         i = queue.pop(0)
29
30         if first_run[i] == -1:
31             first_run[i] = current_time
32             response[i] = first_run[i] - arrival[i]
33
34         # Run for at most one time quantum (preemption point)
35         exec_time = min(quantum, remaining[i])
36         start = current_time
37         current_time += exec_time
38         remaining[i] -= exec_time
39
40         if gantt and gantt[-1][0] == pid[i] and gantt[-1][2] == start:
41             gantt[-1] = (pid[i], gantt[-1][1], current_time)
42         else:
43             gantt.append((pid[i], start, current_time))
44
45         # Newly arrived processes enter queue BEFORE current process re-joins
46         while ptr < n and arrival[order[ptr]] <= current_time:
47             queue.append(order[ptr])
48             ptr += 1
49
50         if remaining[i] == 0:
51             completed += 1
52             completion[i] = current_time
53             turnaround[i] = completion[i] - arrival[i]
54             waiting[i] = turnaround[i] - burst[i]
55         else:
56             queue.append(i)
57
58     return {
59         'pid': pid, 'arrival': arrival, 'burst': burst,
60         'completion': completion, 'turnaround': turnaround,
61         'waiting': waiting, 'response': response, 'gantt': gantt,
62         'avg_tat': sum(turnaround) / n,
63         'avg_wt': sum(waiting) / n,
64         'avg_rt': sum(response) / n
65     }
66
67
68 def display(r, name, quantum=None):
69     title = name if quantum is None else f"{name} (Time Quantum = {quantum})"
70     print(f"==== {title} =====")
71     print("Process\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\tResponse")
72     for i in range(len(r['pid'])):
73         print(f"{r['pid'][i]}\t{r['arrival'][i]}\t{r['burst'][i]}\t"
74               f"{r['completion'][i]}\t{r['turnaround'][i]}\t{r['waiting'][i]}\t"
75               f"{r['response'][i]}")
76     print("Gantt Chart:")
77     for p, s, e in r['gantt']:
78         print(f"[{p}: {s}->{e}]", end=" ")
79     print(f"\nAverage Turnaround Time: {round(r['avg_tat'], 2)}")
80     print(f"Average Waiting Time: {round(r['avg_wt'], 2)}")
81     print(f"Average Response Time: {round(r['avg_rt'], 2)}")
82
83
84 parameters = [
85     ('P1', 0, 7), ('P2', 1, 4), ('P3', 2, 15),
86     ('P4', 3, 11), ('P5', 4, 20), ('P6', 4, 9)
87 ]
88
89 display(round_robin(parameters, 5), "Round Robin", 5)

```

(For SRTF/preemptive)-

```
def srtf(processes):
    n = len(processes)
    pid = [p[0] for p in processes]
    arrival = [p[1] for p in processes]
    burst = [p[2] for p in processes]
    remaining = burst.copy()

    completion = [0] * n
    turnaround = [0] * n
    waiting = [0] * n
    response = [0] * n
    start_time = [-1] * n
    is_completed = [False] * n

    current_time = 0
    completed = 0
    gantt_chart = []
    previous_pid = None

    while completed < n:
        shortest_idx = -1
        min_remaining = float('inf')

        for i in range(n):
            if (arrival[i] <= current_time
                and not is_completed[i]
                and remaining[i] < min_remaining):
                min_remaining = remaining[i]
                shortest_idx = i

        # CPU idle
        if shortest_idx == -1:
            if previous_pid != "IDLE":
                gantt_chart.append(("IDLE", current_time, current_time + 1))
            previous_pid = "IDLE"
            current_time += 1
            continue

        if start_time[shortest_idx] == -1:
            start_time[shortest_idx] = current_time

        current_time += 1 # run 1 unit, then re-check
        remaining[shortest_idx] -= 1

        if previous_pid != pid[shortest_idx]:
            gantt_chart.append((pid[shortest_idx], current_time - 1, current_time))
```

else:

gantt_chart[-1] = (pid[shortest_idx], gantt_chart[-1][1], current_time)

previous_pid = pid[shortest_idx]

if remaining[shortest_idx] == 0:

is_completed[shortest_idx] = True

completion[shortest_idx] = current_time

turnaround[shortest_idx] = completion[shortest_idx] - arrival[shortest_idx]

waiting[shortest_idx] = turnaround[shortest_idx] - burst[shortest_idx]

completed += 1

for i in range(n):

response[i] = start_time[i] - arrival[i]

return {'pid': pid, 'arrival': arrival, 'burst': burst,

'completion': completion, 'turnaround': turnaround,

'waiting': waiting, 'response': response,

'gantt_chart': gantt_chart}

processes = [

('P1', 0, 7), ('P2', 1, 4), ('P3', 2, 15),

('P4', 3, 11), ('P5', 4, 20), ('P6', 4, 9)

]

result = srtf(processes)

n = len(processes)

print("==== SRTF (Preemptive SJF) =====")

print("Process\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\tResponse")

for i in range(n):

print(f"{result['pid'][i]}\t{result['arrival'][i]}\t{result['burst'][i]}\t"

f"{result['completion'][i]}\t{result['turnaround'][i]}\t"

f"{result['waiting'][i]}\t{result['response'][i]}")

print("Gantt Chart:")

for name, start, end in result['gantt_chart']:

print(f"[{name}: {start}->{end}]", end=" ")

print()

print(f"\nAverage Turnaround Time: {round(sum(result['turnaround']) / n, 2)}")

print(f"Average Waiting Time: {round(sum(result['waiting']) / n, 2)}")

print(f"Average Response Time: {round(sum(result['response']) / n, 2)}")

Output Screenshot :

Round Robin (preemptive)

```
• masfirahman CSE-402-Operating-System-Lab- 23:08 /usr/local/bin/python3 "/Users/masfirahman/Desktop/CSE-402-Operating-System-Lab-/Round Robin/Round Robin (preemptive).py"
===== Round Robin (Time Quantum = 5) =====
Process Arrival Burst Completion Turnaround Waiting Response
P1 0 7 31 31 24 0
P2 1 4 9 8 4 4
P3 2 15 55 53 38 7
P4 3 11 56 53 42 11
P5 4 20 66 62 42 15
P6 4 9 50 46 37 20
Gantt Chart:
[P1: 0->5] [P2: 5->9] [P3: 9->14] [P4: 14->19] [P5: 19->24] [P6: 24->29] [P1: 29->31] [P3: 31->36] [P4: 36->41] [
P5: 41->46] [P6: 46->50] [P3: 50->55] [P4: 55->56] [P5: 56->66]
Average Turnaround Time: 42.17
Average Waiting Time: 31.17
Average Response Time: 9.5
```

(for SRTF/preemptive)

```
• masfirahman CSE-402-Operating-System-Lab- 23:08 /usr/local/bin/python3 "/Users/masfirahman/Desktop/CSE-402-Operating-System-Lab-/Round Robin/srtf.py"
===== SRTF (Preemptive SJF) =====
Process Arrival Burst Completion Turnaround Waiting Response
P1 0 7 11 11 4 0
P2 1 4 5 4 0 0
P3 2 15 46 44 29 29
P4 3 11 31 28 17 17
P5 4 20 66 62 42 42
P6 4 9 20 16 7 7
Gantt Chart:
[P1: 0->1] [P2: 1->5] [P1: 5->11] [P6: 11->20] [P4: 20->31] [P3: 31->46] [P5: 46->66]
Average Turnaround Time: 27.5
Average Waiting Time: 16.5
Average Response Time: 15.83
```

Comparison between Round Robin (preemptive) & SJF (preemptive) :

Numeric :

Metric	Round Robin	SRTF
Average TAT	42.17	27.15
Average WT	31.17	16.5
Average RT	9.5	6.0

Feature Comparison :

Aspect	SJF (preemptive)	Round Robin
Selection	Shortest Remaining Burst Time	Order of arrival (cyclic)
Preemption Trigger	Shorter Job arrives	Time Quantum Expires
Starvation	Possible	Not Possible
Average Waiting Time	Minimum (optimal)	Higher than SRTF
Fairness	Not fair (prefers short jobs)	Fair
Use Case	Batch System, theoretical benchmark	Time Sharing/interactive system

Why SRTF (preemptive SJF) is better (efficiency perspective):

1. **Provably minimum waiting time:** SRTF always runs the job closest to completion, which greedily minimizes total waiting. On this input it cuts average WT nearly in half (16.5 vs 31.17).
2. **Fewer wasteful context switches:** RR switches on every quantum expiry whether it helps or not (13 switches here); SRTF switches only when beneficial (6 switches), so less CPU time is lost to overhead.
3. **Short jobs finish fast:** P2 completes at $t=5$ under SRTF but $t=9$ under RR; quick completion of short jobs empties the ready queue faster and improves throughput.
4. **No quantum-tuning problem:** RR's performance depends heavily on the quantum (too large $\rightarrow$ behaves like FCFS; too small $\rightarrow$ overhead dominates). SRTF has no such parameter to tune.
5. **Better turnaround overall:** Average TAT drops from 42.17 (RR) to 27.5 (SRTF) on identical input.

Discussion :

The experimental outputs transform the theoretical fairness-efficiency trade-off into a quantifiable reality. Under Round Robin ($TQ = 5$), the scheduler enforces strict egalitarianism: every process receives its first CPU slice by $t=24$, and the worst first-response delay is capped at 20 units (P6). Conversely, SRTF operates on a utilitarian principle; short jobs respond instantaneously (P2's response = 0), but long jobs are systematically deprioritized. P5 remains locked out until $t=46$, yielding a response time of 42—nearly three times worse than its RR counterpart. Consequently, RR's average response time (9.5) outperforms SRTF's (15.83), even as SRTF decisively wins on waiting time (16.5 vs. 31.17) and turnaround time (27.5 vs. 42.17). This proves that the two algorithms do not represent a hierarchy of quality, but rather optimize for fundamentally different operational goals.

A critical secondary observation is the hidden cost of context switching. The RR Gantt chart exhibits 14 execution segments (13 context switches), several of which represent pure overhead. For instance, P4 is

preempted at $t=41$ with only 1 unit of burst remaining, and P5's final two quanta run back-to-back, rendering the switch at $t=61$ entirely wasteful. SRTF completes the identical workload with only 7 segments (6 switches), switching only when a strictly shorter job arrives. RR pays a continuous "overhead tax" upon every quantum expiry, whereas SRTF's preemptions are strictly purposeful.

Finally, the experiment highlights the distinct vulnerabilities of each approach. RR suffers from parameter sensitivity: a quantum of 20+ degrades it into FCFS, while a quantum of 1–2 causes thrashing via excessive context switching. SRTF, while parameter-free, suffers from informational dependency—it requires exact burst times in advance, which is rarely feasible outside of batch systems. In practical software engineering, such as managing asynchronous task queues in browser extensions (e.g., balancing UI rendering threads against background data-fetching jobs), this exact trade-off manifests. RR-style rotation keeps the user interface responsive, while SRTF-style prioritization minimizes the total completion time of background tasks.

Conclusion :

This lab demonstrates that the concept of a "better" scheduling algorithm is entirely metric-dependent. Round Robin yields higher average waiting and turnaround times but provides a superior, evenly distributed response time and a mathematical guarantee against starvation. SRTF achieves near-optimal waiting and turnaround times but at the severe cost of starving long-running processes and requiring foreknowledge of burst lengths.

Therefore, Round Robin remains the superior default for interactive, time-sharing environments where user-perceived responsiveness and fairness are paramount. SRTF serves as the theoretical efficiency benchmark for offline, batch-processing workloads where burst lengths are known and throughput is the sole priority. Ultimately, the results explain why modern, practical operating systems avoid pure implementations of either. Instead, they utilize hybrid architectures like Multilevel Feedback Queues (MLFQ), which rotate processes in a Round Robin fashion while dynamically promoting or demoting them based on observed CPU burst behavior. This hybrid approach successfully captures the fairness and responsiveness of RR without entirely sacrificing the throughput efficiency of SRTF.

Github repository : <https://github.com/MasfiRahman/CSE-402-Operating-System-Lab-.git>